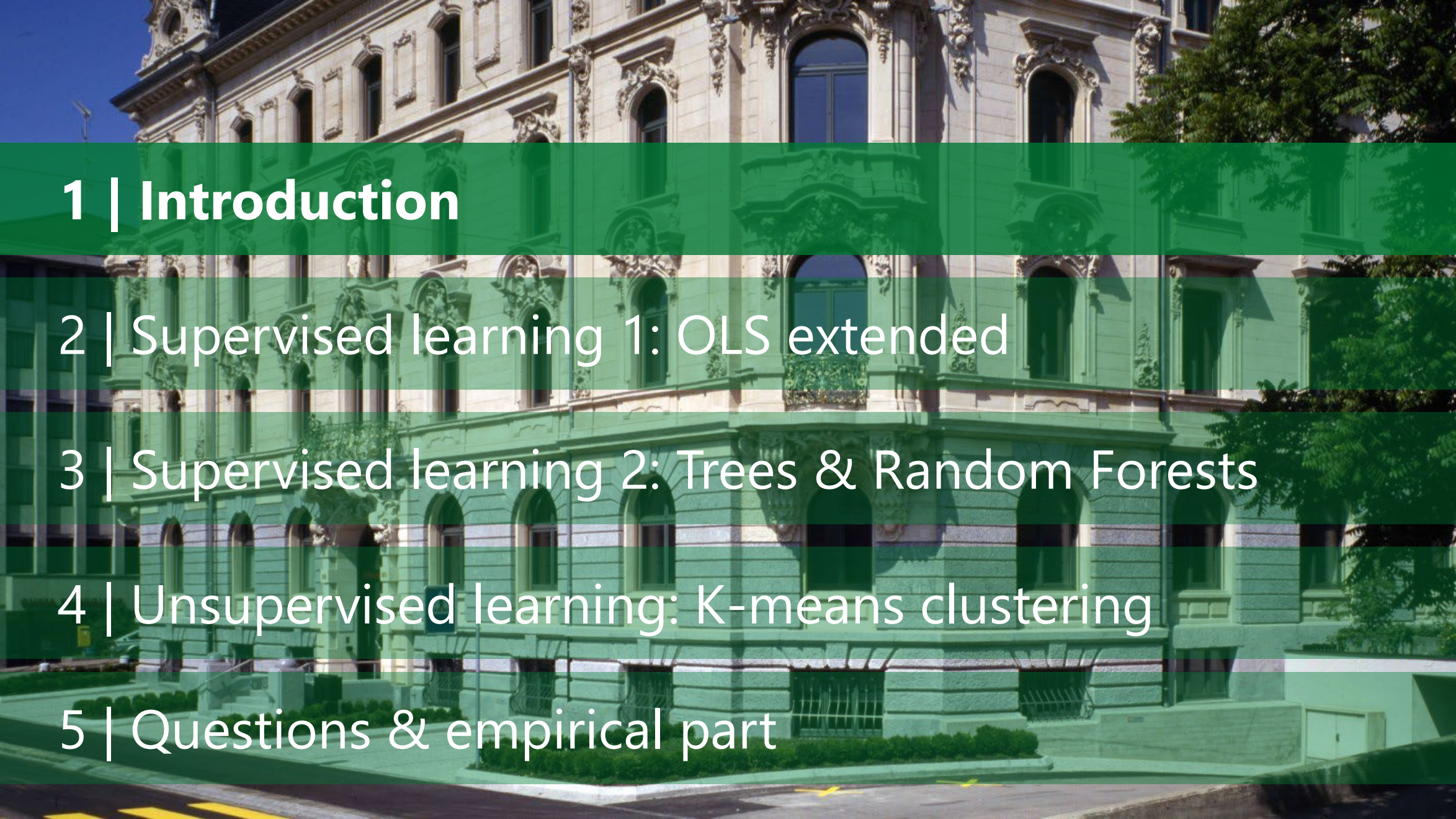
Causal Machine Learning & Artificial Intelligence for Economists

Michael Lechner
Swiss Institute for Empirical Economic Research (SEW-HSG)
University of St. Gallen | Switzerland

Lecture 2: Machine Learning (for prediction)

Required new external Python modules (from PyPI): scikit-learn

Disclaimer: Almost all figures created by ChatGPT



1 | Introduction

2 | Supervised learning 1: OLS extended

3 | Supervised learning 2: Trees & Random Forests

4 | Unsupervised learning: K-means clustering

5 | Questions & empirical part



Machine Learning | 1

What is *Machine Learning*?

- Everything & nothing ➔ Here: *Statistical Learning*
- Flexible prediction methods

Types of ML

- **Supervised** (*y, x)* & *unsupervised* learning (*x*)
  - Supervised: Classification (discrete *y*) & **regression** (cond. expectations)



Supervised & unsupervised learning

Supervised Learning: Find structure in *X* that allows 'good' prediction of *Y*

- If variable in *X* is important for *Y*: Similar values of this variable will be related to similar values of *Y*
- Comparing predictions with realisations of *Y* provides metric for optimal combination of features (*X*)
  (Y is 'teacher'; Y 'tells' algorithm how 'good' its prediction is, based on its current combination of X)
- Regression (classification)-type methods
- Examples of supervised, regression ML (details later)
  - All classical econometric estimators
  - Regularised, shrinkage estimators (Lasso, Ridge, ELN, …)
  - Neural Networks
  - *Trees & Random Forests*

Unsupervised learning: Find similarity of *X* without having access to *Y*

- Clustering etc.



Supervised statistical learning | 1

Standard estimation problem: $y = f(x) + e$

- Task Estimate unknown (possibly non-linear) function *f(.)* from data
  - Example: OLS ($f(x) = x\hat{\beta}$)
  - Statistical learning methods provide alternative approaches that may be more powerful by
    - allowing for non-linearities
    - allowing for very many features
    - Using computationally efficient algorithms



Supervised statistical learning | 2 | Avoid overfitting !

The prediction error $y_i - \hat{f}(x_i)$ can be reduced in the *training data* by making the model more flexible

But: Bad idea to use the same data to judge quality of estimation in the population or another (future) sample → *overfitting*

Divide the (large) data!

- *Training data*: Sample used for estimating *f(x)*
- *Test and/or validation data*: Different data to evaluate performance of estimator 'out-of-sample'

Good ML is not fitting the past (training data), but it is predicting for new data



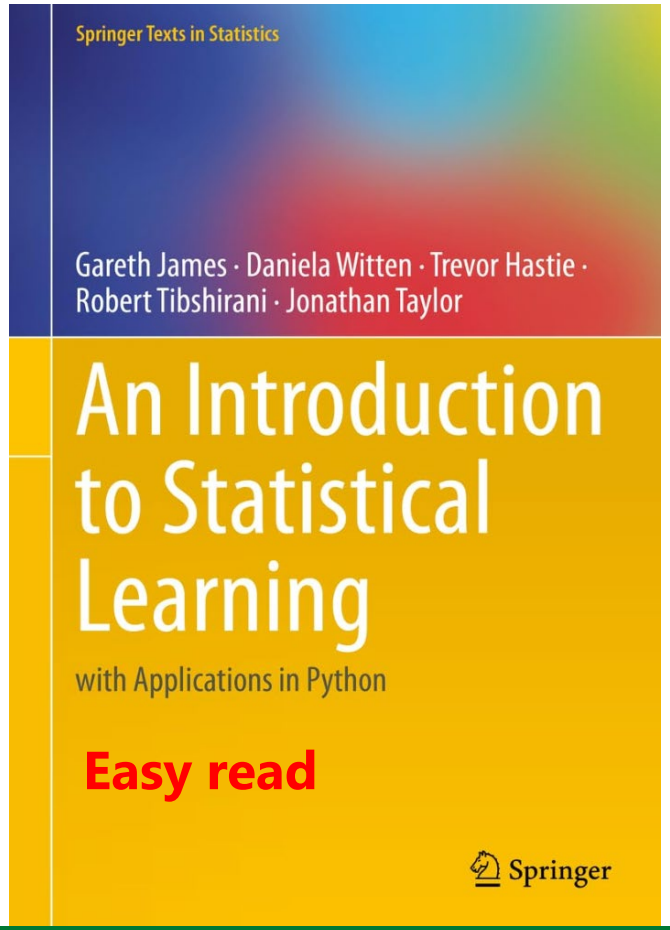
Supervised statistical learning | 3

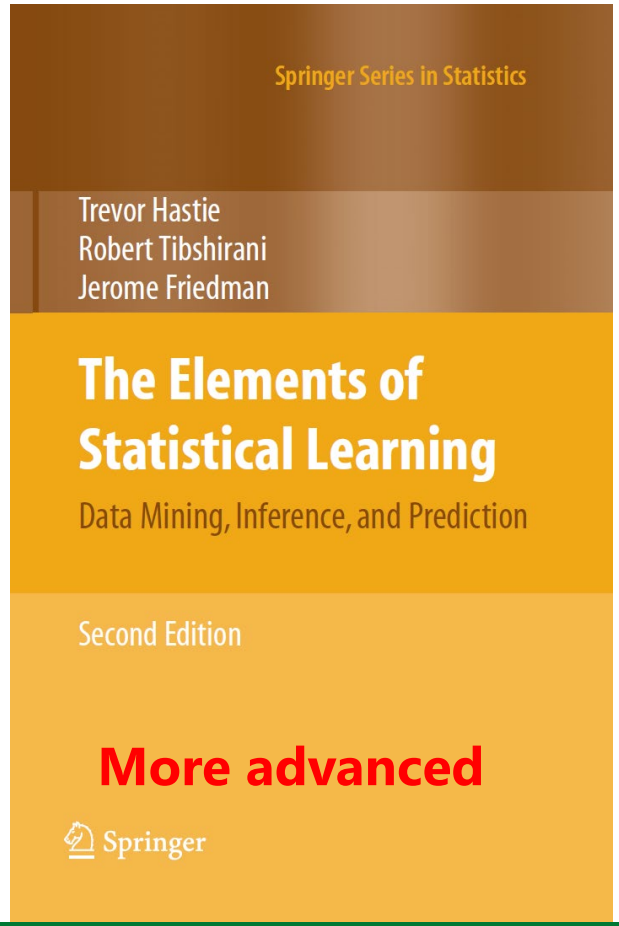
Why do economists care about supervised learning at all?

- Get predictions of key economic indicators given current policy & environment
- Some causal methods (discussed later) require good predictions as inputs into the particular causal estimator



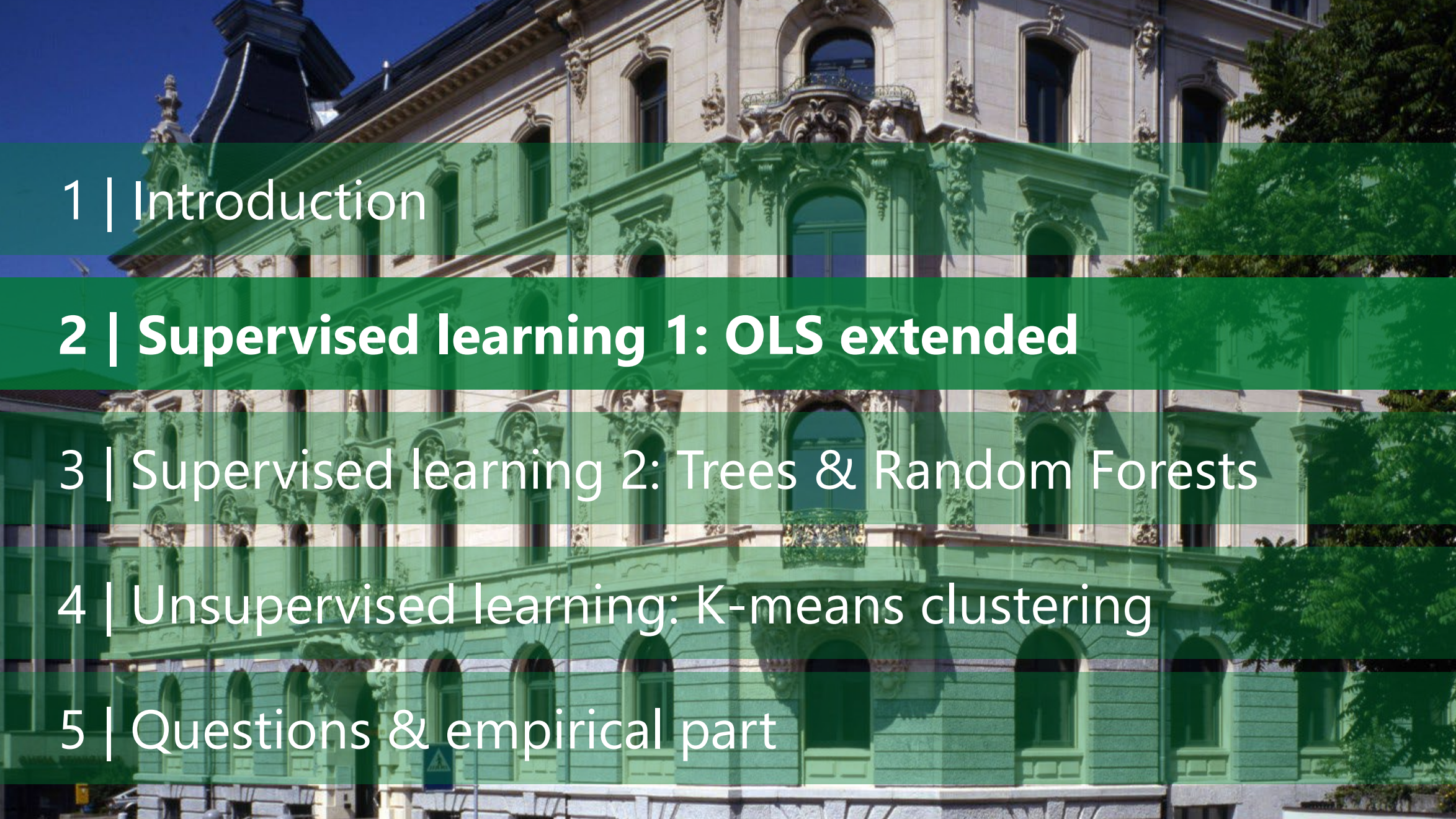
Literature


Springer Texts in Statistics
Gareth James · Daniela Witten · Trevor Hastie ·
Robert Tibshirani · Jonathan Taylor
An Introduction to Statistical Learning
with Applications in Python
Easy read
Springer


Springer Series in Statistics
Trevor Hastie
Robert Tibshirani
Jerome Friedman
The Elements of Statistical Learning
Data Mining, Inference, and Prediction
Second Edition
More advanced
Springer







1 | Introduction

2 | Supervised learning 1: OLS extended

3 | Supervised learning 2: Trees & Random Forests

4 | Unsupervised learning: K-means clustering

5 | Questions & empirical part



Shrinkage Methods

Shrinkage methods are based on classical regression estimators (e.g., OLS) but with penalty terms penalising large coefficient values

Methods differ in the way they specify the penalty term

- **L**east **A**bsolute **S**hrinkage & **S**election **O**perator: λ * sum of absolute values of coefficients
- **Ridge** regression: λ * sum of squared values of coefficients
- **Elastic net**: λ_1 * sum of absolute values of coefficients + λ_2 * sum of squared values of coefficients



Ridge regression, LASSO, Elastic Net | 1

$$\hat{\beta}^{Ridge} = \arg\min_{\beta} \sum_{i=1}^{N} \left(y_i - x_i\beta\right)^2 + \lambda \sum_{j=1}^{p} \beta_j^2; \quad \lambda \geq 0$$

$$\hat{\beta}^{Lasso} = \arg\min_{\beta} \sum_{i=1}^{N} \left(y_i - x_i\beta\right)^2 + \lambda \sum_{j=1}^{p} \left|\beta_j\right|; \quad \lambda \geq 0$$

$$\hat{\beta}^{ElasticNet} = \arg\min_{\beta} \sum_{i=1}^{N} \left(y_i - x_i\beta\right)^2 + \lambda_1 \sum_{j=1}^{p} \left|\beta_j\right| + \lambda_2 \sum_{j=1}^{p} \beta_j^2; \quad \lambda_1 \geq 0, \quad \lambda_2 \geq 0$$

λ : penalty multiplier

- If $\lambda = 0$: OLS
- If $\lambda \rightarrow \infty$: Only constant term remains in model (coefficient of constant term is not penalised)



Ridge regression, LASSO, Elastic Net | 2

Advantages of shrinkage methods for prediction

- More flexible than OLS ➔ potentially improved predictive power
- Ultra-high dimensional covariates are possible
  - UHD: *dim(X)* > *N*

Estimates of coefficients are shrunken (usually) towards zero

- This leads to (usually downward) biased coefficient estimators

Gauss–Markov does not imply that OLS dominates shrinkage estimators: shrinkage introduces bias in exchange for lower variance & potentially lower prediction error



Ridge regression, LASSO, Elastic Net | 3 | *Penalty term*

Choice of penalty is related to bias-variance trade-off (***MSE = Variance + Bias²***)

- Increasing penalty term (λ) increases bias & reduces variance
  - If $\lambda = 0$: OLS (unbiased but noisy)
  - $\lambda \rightarrow \infty$: Prediction is average of Y for everybody (very precise but very biased)

Choose penalty term that leads to best predictive performance (e.g., in terms of MSE)

- Optimal value of λ determined by grid search (cross-validation)
- Minimise out-of-sample MSE

$$\hat{\lambda} = \arg\min_{\lambda} \hat{MSE}(\lambda) = \arg\min_{\lambda} \frac{1}{N} \sum_{i=1}^{N} y_i - \hat{y}(x_i; \lambda), \qquad \hat{y}(x_i, \lambda) = \hat{f}_{-i}(x_i; \lambda)$$

- $\hat{f}_{-i}$: i is not used in the estimating of the prediction function (out-of-sample prediction)



Remark: How does **Cross-Validation (CV)** work?

k-fold CV is a method to create out-of-sample predictions without loss of training data

1) Randomly split data in *k* subsamples of equal size
2) Combine *k*-1 subsamples to train model (obtain $\hat{f}_{-i}$)
3) Predict values for sample not used in step 2
4) Repeat steps 2) and 3) for all *k* combinations (i.e., until every observation has been predicted once out of sample)
5) Choose the tuning parameter with the best CV performance and re-estimate the final model using all training data



Ridge regression, LASSO, Elastic Net | 4

Which one to use in practice?

- Sparse models (few X belong to true model) → LASSO
- Dense models (many X belong to true model) → Ridge regression
- Unknown (or in-between) level of sparsity → Elastic Net

Final notes

- LASSO is special: Estimated coefficients may be exactly 0.000000
- General: Shrinkage methods accept bias in coefficients & predictions to reduce noise
- Estimators are sensitive to the scaling of X → normalise continuous X to mean = 0, variance = 1



Neural networks

Here: Only **N**eural **N**etworks for regression type learning problems

Marketing type of definition

- Flexible Learning Model achieving good prediction properties by using a model structure that is like the human brain

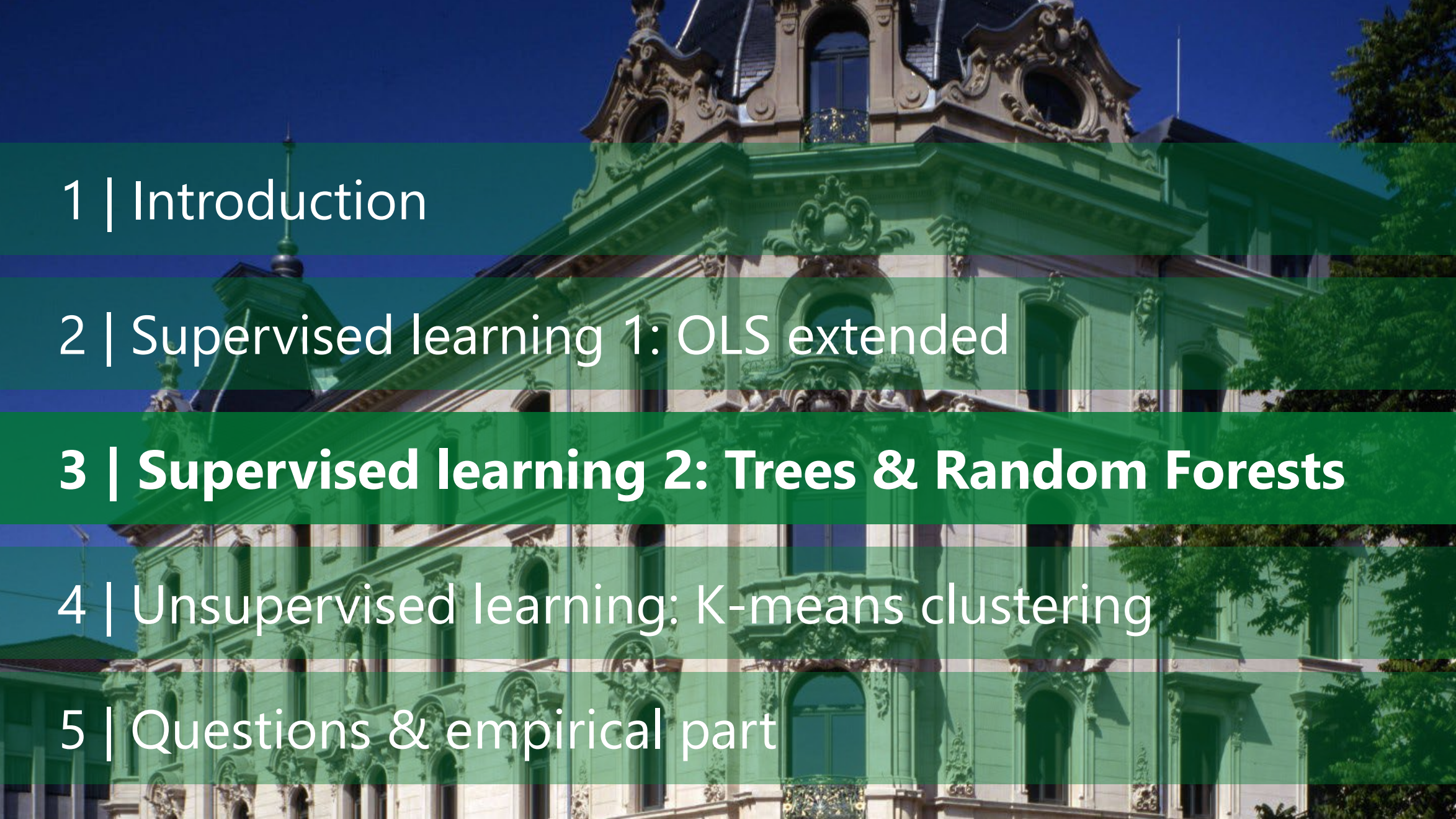
Statistical definition

- Flexible, regularised, global, non-linear estimator

Theoretical justification: Universal approximation theorem suggests that a flexible NN can approximate a very broad class of nonlinear functions arbitrarily well







1 | Introduction

2 | Supervised learning 1: OLS extended

3 | Supervised learning 2: Trees & Random Forests

4 | Unsupervised learning: K-means clustering

5 | Questions & empirical part



Regression Trees





Concept

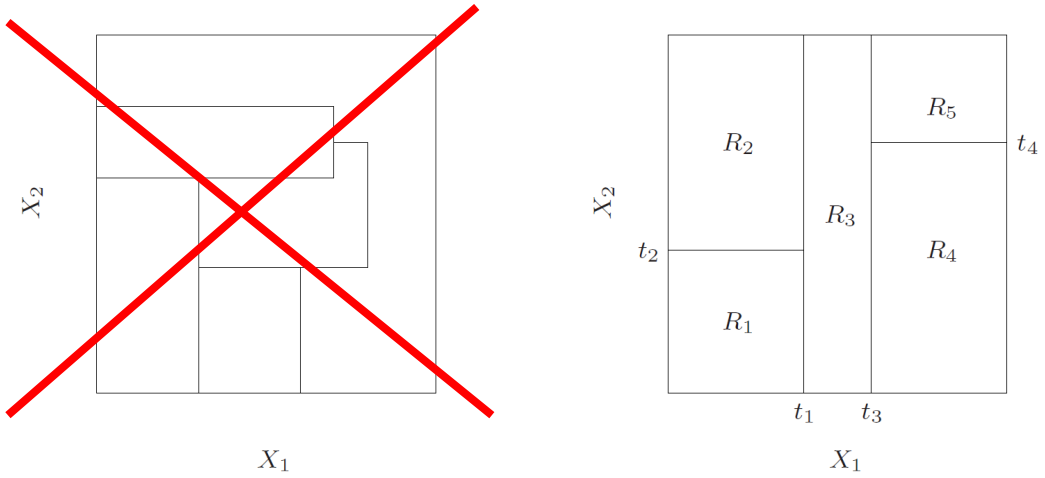
$$y_i = f(x_i) + e_i \qquad \hat{y}_i = \hat{f}(x_i)$$

Main idea

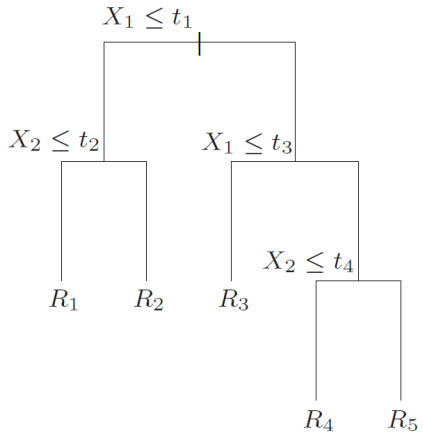
- Stratify data in distinct strata (leaves) with similar values of *x*
- Mean of *y* in leaf is predictor for *y* for all observation with values of *x* that belong to this leaf

Build tree by further splitting existing strata with recursive binary splits

- In each stratum (=leaf), for each feature in *X*, & each of its values, split old stratum into two new strata such that prediction error (e.g., MSE) of the new prediction is minimised



The splitting rule



Estimate of *f(x)*

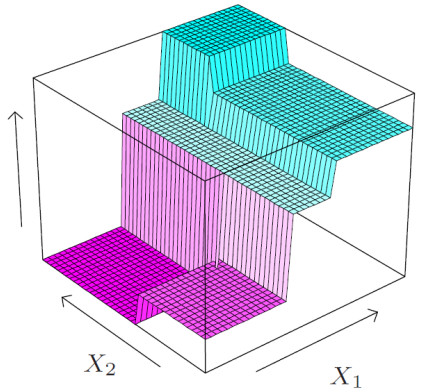


FIGURE 9.2. *Partitions and CART. Top right panel shows a partition of a two-dimensional feature space by recursive binary splitting, as used in CART, applied to some fake data. Top left panel shows a general partition that cannot be obtained from recursive binary splitting. Bottom left panel shows the tree corresponding to the partition in the top right panel, and a perspective plot of the prediction surface appears in the bottom right panel.*



Illustration of a tree with two continuous covariates

Hastie et al. (2013), p. 306



Growing the tree | 3

The variance-bias trade-off

- If tree is too large → overfitting → too much variance
- If tree is too small → not precise enough → too much bias

Find stopping rule that prevents overfitting



Regression vs. classification trees

Regression vs. classification

- Quality of fit measure (objective function) & predictor may be different

Regression tree

- Goal is to approximate *f(x)* by a conditional expectation *E(y|x)*
- Taking averages in leaf as prediction & MSE as quality measure is 'natural'

Classification tree

- If *y* is discrete (& multivalued & unordered), an approximation of *f(x)* by *E(y|x)* is not useful
- '*Majority rule*' to obtain prediction
- Alternative quality-of-prediction measure required



Pros & cons of trees

Pros

- Very flexible
  - they capture substantial non-linearities
- Nice interpretation of leaves in terms of sets of values of *x*
- Easy to understand for non-statisticians (easy to plot)

Cons

- Computation increases with p, because many candidate variables/splits must be evaluated
- They may be very unstable (path dependence) → slight change in sample may lead to very different tree ➔ loss of predictive power compared to other methods



Bagging trees | 1

Single trees tend to be unstable → delete 1 obs → likely to get different trees → likely to get unstable predictions

Idea of **b**ootstrap-**a**vera**ging** **b**agging

- Make use of this instability: Average many unstable trees to stabilise predictions

Idea of bagging

- Build a *deep* tree ➔ small final leaves ➔ low bias, large variance of leaf averages
- Do this for many bootstrap samples
  - Bootstrap sample is obtained by drawing *N*-times in original data (with replacement)
- Average the predictions over these trees ➔ variance reduction

The result is less noisy predictions while keeping bias low



Bagged trees | 2

Advantages compared to single tree

- Variance reduction → better predictive properties

Disadvantages

- Increased computation time
  - *B* determined such that *Out-of-bag* MSE stabilises (or *B* predetermined)
- Loss of simple interpretation
  - there is no single tree that represents the resulting prediction rule



Random Forests





Random Forests

Special Bagged Tree (used for RF)

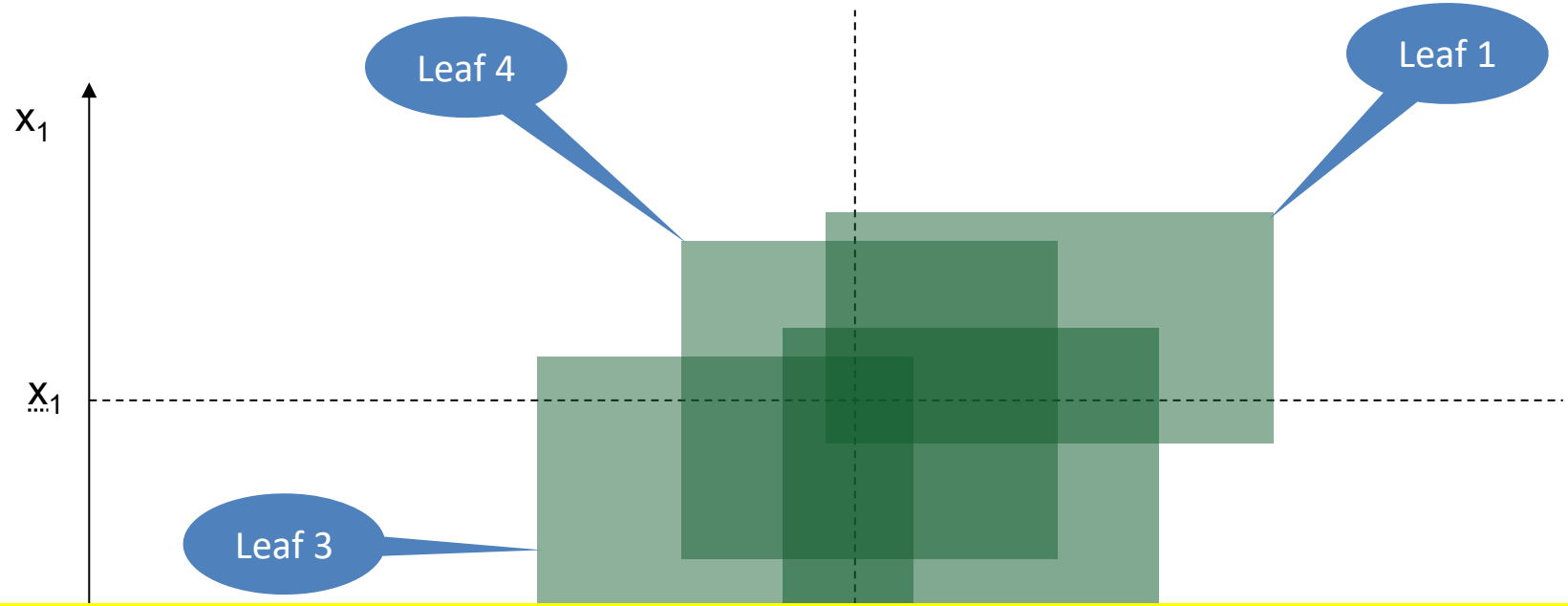
- When deciding on the next split, only a random selection of variables is evaluated
- This decorrelates the bagged trees & reduces computation

Prediction of a Random Forest is the mean of the predictions of many such trees

- Computationally efficient & very effective in non-linear settings



Bagged trees & RF are weighting estimators of *f(x)*

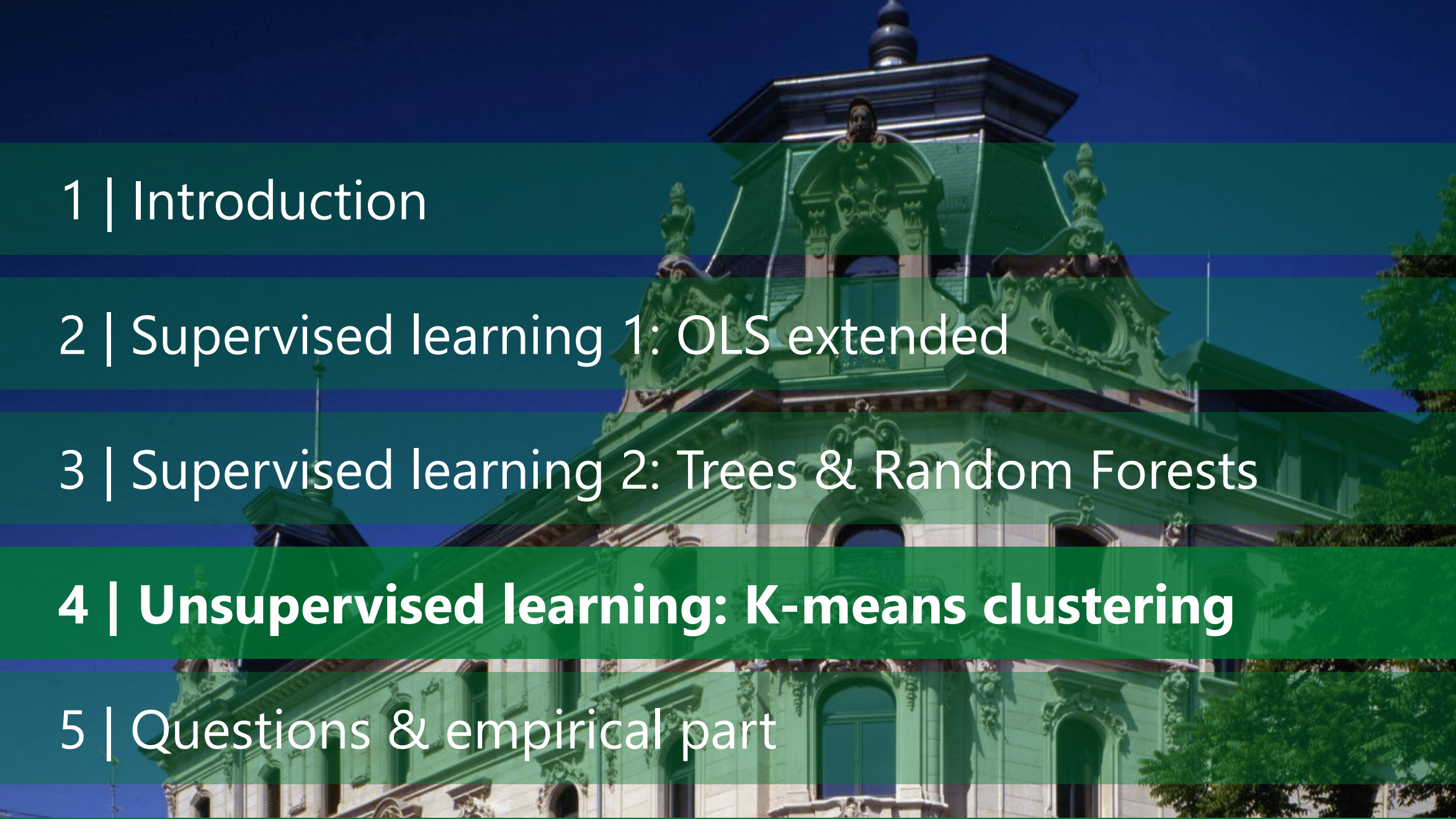


Weighting:
(i) Each tree predicts by the mean of y_i in the terminal leaf containing x
(ii) The forest averages these predictions across trees
(iii) Equivalently, the RF prediction is a weighted average of the observed y_i's

Data-adaptive neighbourhoods: the regions defining the weights adapt to both the data distribution & the variables relevant for prediction (i.e., the implied bandwidth adapts)







1 | Introduction

2 | Supervised learning 1: OLS extended

3 | Supervised learning 2: Trees & Random Forests

4 | Unsupervised learning: K-means clustering

5 | Questions & empirical part



Introduction | 1

Unsupervised learning searches for structure in X without using an outcome Y

- It can help summarise, describe, or reduce the complexity of data

Many different methods exist

- Here, we briefly discuss one cluster method (k-means) as example



Intro & notation

K-means clustering is very popular because of its

- Simplicity (& computational efficiency)

Squared Euclidean distance used to measure similarity: $d(x_i, x_{i'}) = \sum_{j=1}^{p} (x_{ij} - x_{i'j})^2 = \|x_i - x_{i'}\|^2$

Find clusters such that observations are most similar within clusters & most different across clusters



The algorithm

1) Set number of clusters to *C*

2) Designate *C* data points as (starting) cluster centres (randomly or more sophisticated)

3) Assign each observation to its closest cluster centre

4) Compute new cluster centres as mean of *x*-values of observations in each cluster

Repeat 3) and 4) until no observation changes cluster in step 3)

Note

- Algorithm can be sensitive to scaling of X (normalise!) & starting values



Simulation | DGP

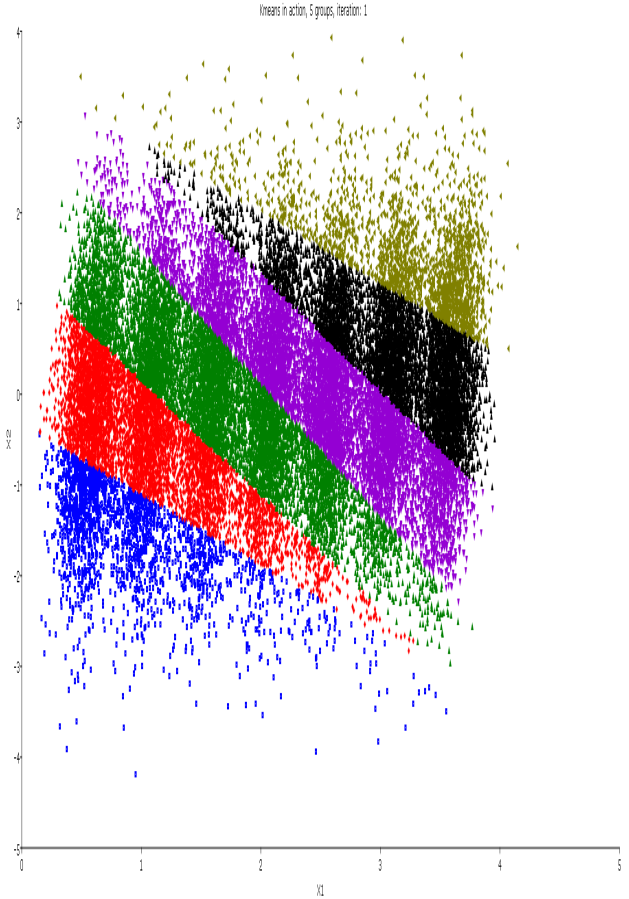
$$X \sim N\left(\begin{pmatrix}0\\ \mu\end{pmatrix}, \begin{pmatrix}1 & 0.5\\ 0.5 & 2\end{pmatrix}\right); \quad \mu_i \in [1,2,3,4,5,6,7] \text{ (with equal probability)}$$

N=30'000

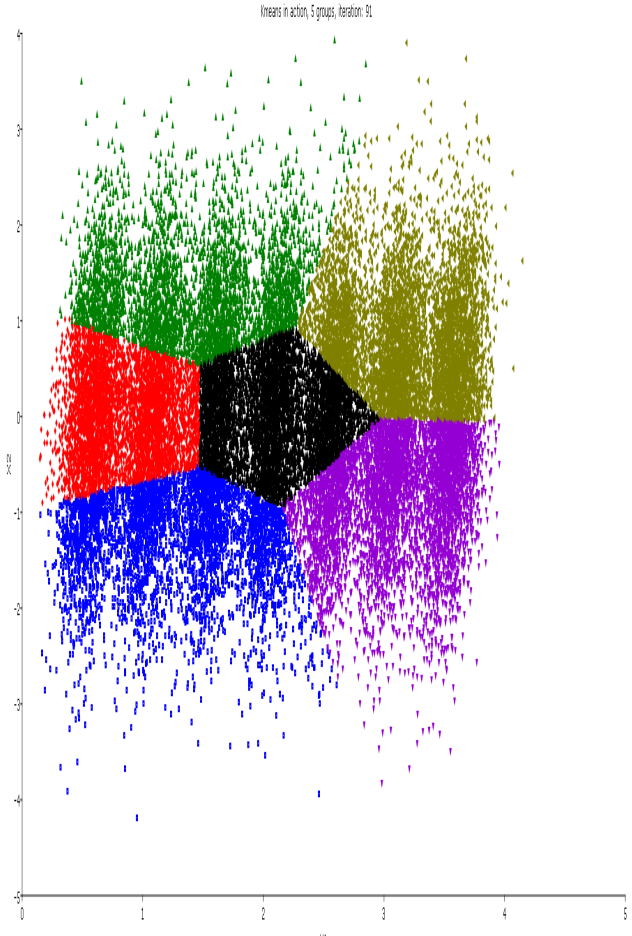
K-means with 6 (below) & 7 (live) groups (no retries)



Simulation | Start

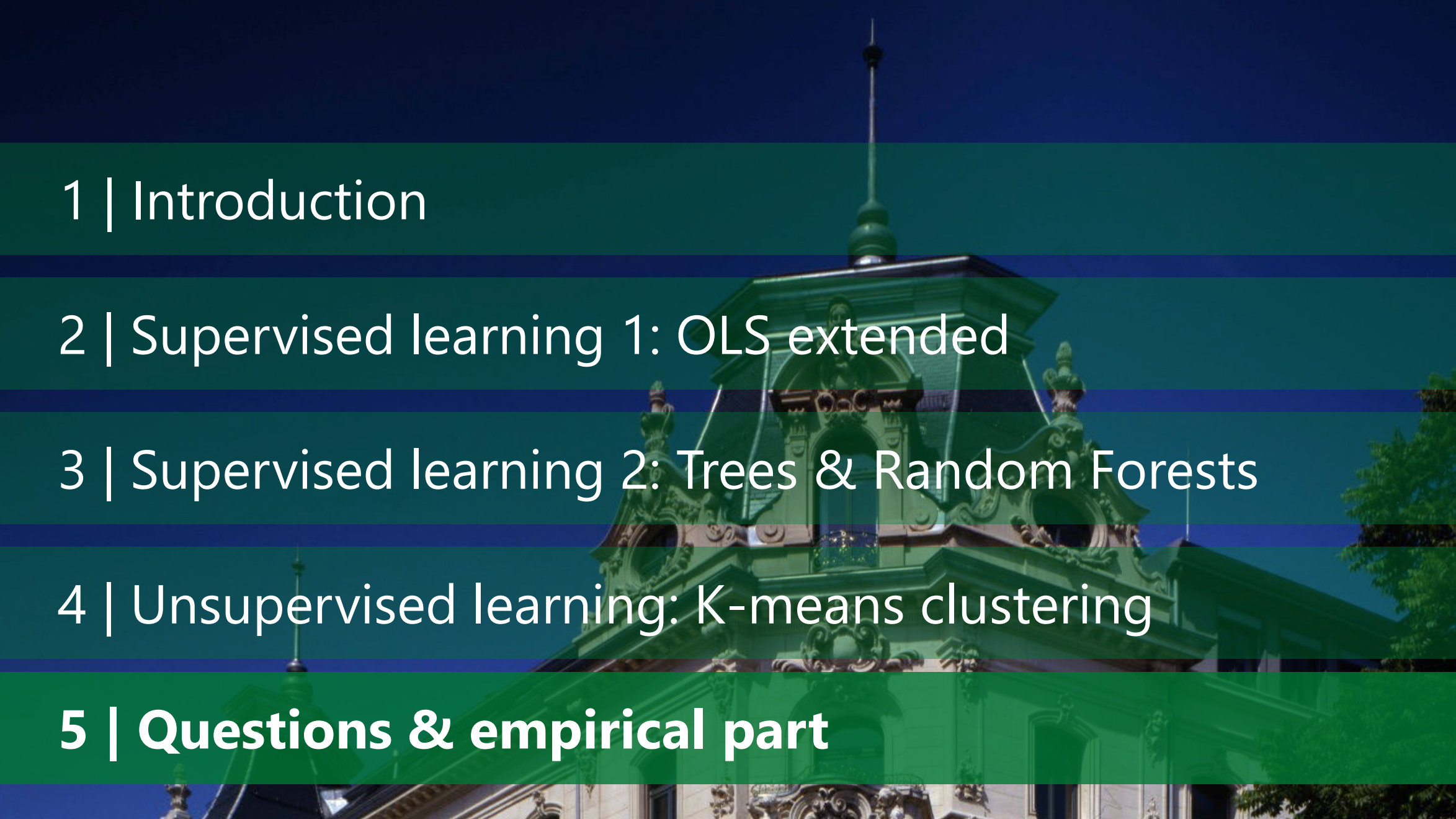


End (92 iterations)









1 | Introduction

2 | Supervised learning 1: OLS extended

3 | Supervised learning 2: Trees & Random Forests

4 | Unsupervised learning: K-means clustering

5 | Questions & empirical part







Rules for group work (including graded group exercise)

Groups may have up to 4 members

- *All group members must agree to all group members*

Group memberships may change until lecture 3

You are free to use any AI you like

- Graded exam: Contact with humans is prohibited & will be monitored (& will be useless )

Lecturer is available for discussions & help during group work

- limited in graded group work

You should always ***install & test*** the required packages ***before the lecture***

- ***Required (additional) packages are always shown on 1st slide of each lecture***



Empirical group work | General

In the next 4 sessions we use the data set *ALMP_data.csv* (on Canvas, zipped)

- This is a cleaned data set collected for the analysis of active labour market policies
- A detailed description of this data is contained in the file *ALMP_data_info.pdf* (on Canvas)

Today, we set up 2 profiling systems

Both systems aim to predict the future unemployment state based on data already known to January, 19X3

The two systems differ only with respect to the measure of unemployment used

- Target 1: EMPLX9_4 == 2 (unemployed in the 4th quarter of 19X9; latest available measure)
- Target 2: Total number of quarters of unemployment in January, 19X3



Empirical group work | Task

Your task is to find the best prediction methods

- Use a classifier for the binary outcome
- Use a regression-type method for the total quarters of unemployment
- You are free to chose among all methods available in scikit-learn
  - Make sure to include only pre-19X3 variables
  - Make sure to obtain a good out-of-sample performance in terms of accuracy (classifier) & mean squared error (regression)
- Your success will be evaluated on new data I will make available at the end of the session
  - Provide code that is available to compute the performance of your preferred estimator on new data

The best group will get a small prize



Next lecture

Causal machine learning (aggregate effects)



Background slides

(for interested students only, not for exam)



Background

Neural networks | 2

Heavily used in modern ML in practice for very many different tasks

Very powerful, but also computationally very demanding

- May need specific hardware architecture to be feasible

Huge number of parameters needed for good performance

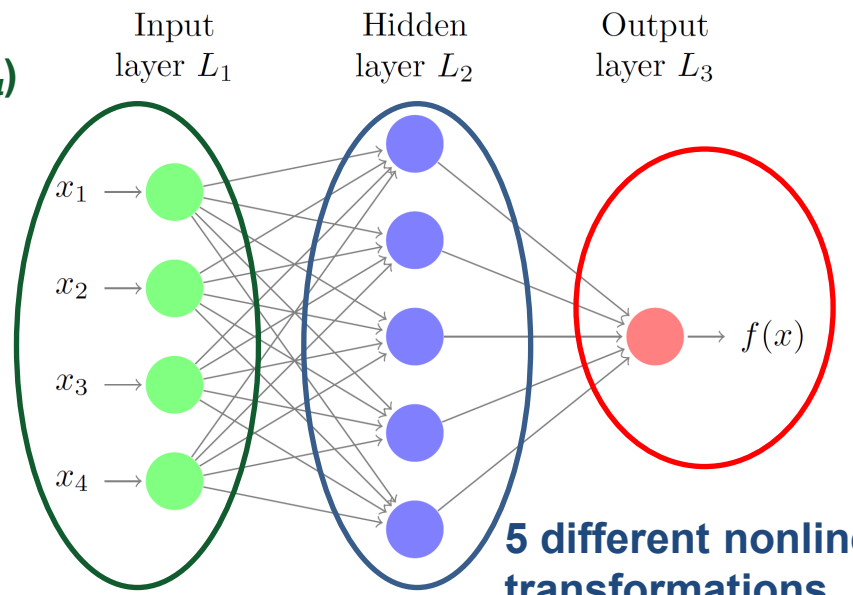
- Tuning very important
  - Choice of architecture
  - Regularization
- 'More art than science'



Background

Graphical representation of a very simple NN

Covariates (X_1, …, X_4)



Outcome specific linear or nonlinear combination of 5 non-linear functions leads to prediction for *Y*

5 different nonlinear transformations

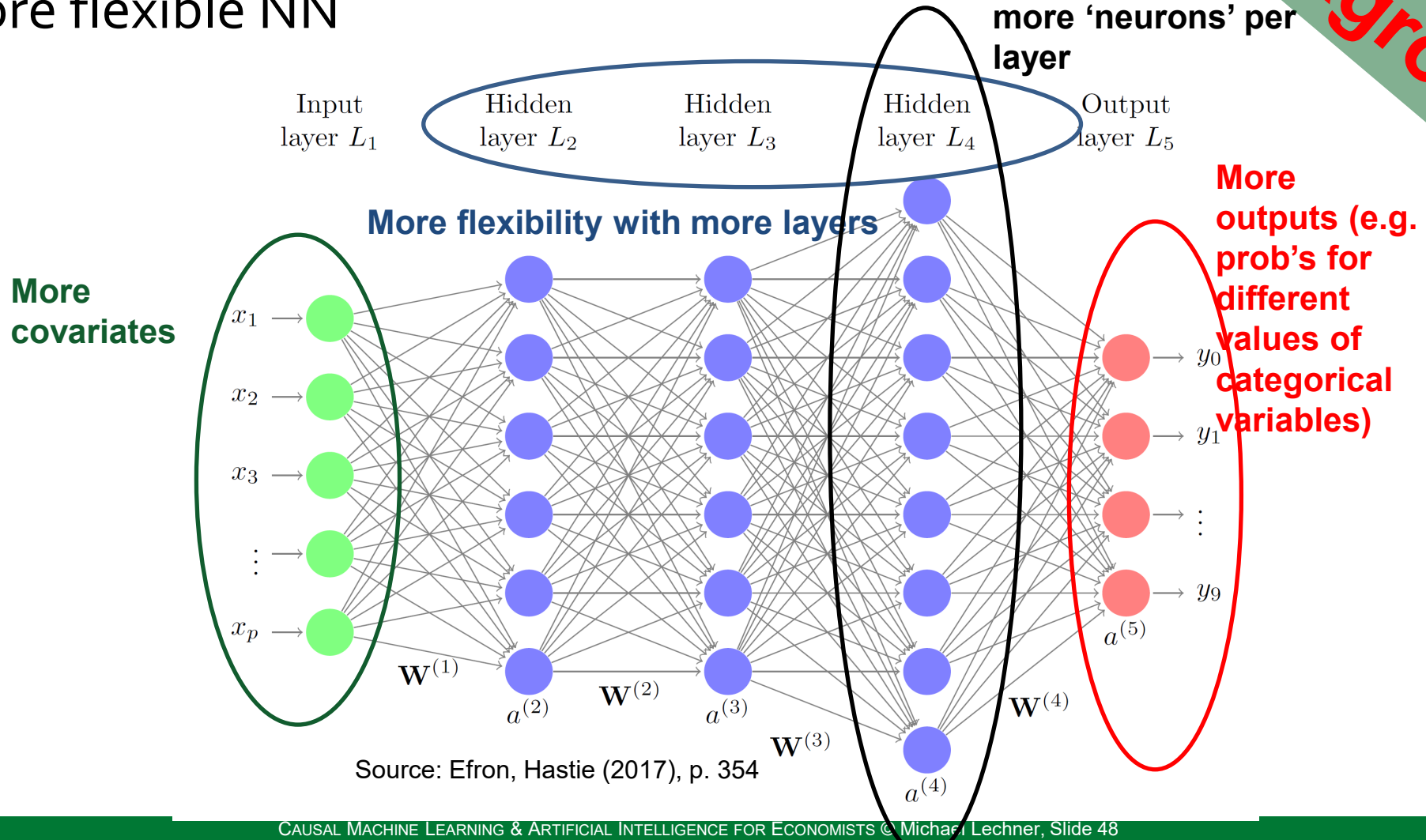
Figure 18.1 Neural network diagram with a single hidden layer. The hidden layer derives transformations of the inputs—nonlinear transformations of linear combinations—which are then used to model the output. Source: Efron, Hastie (2017), p. 351



Background

A more flexible NN

More flexibility with more 'neurons' per layer



Source: Efron, Hastie (2017), p. 354



Background

Neural networks | A1

When writing the statistical model, it is useful to account for the particular architecture in the notation

- P covariates, L layers, K^l neurons in layer l, $z^{nl'}$ output of neuron n in layer l', M possible outcomes

$$z^{k,1}(x) = g^1(\alpha^{k,1} + \sum_{p=1}^{P} x^p \beta^{p,k,1}), \quad l = 1, \quad \forall k^1$$

$$\vdots$$

$$z^{k,l}(x) = g^l(\alpha^{n^1} + \sum_{k=1}^{K^{l-1}} z^{k,l-1} \beta^{k,l}), \quad l > 1, \quad \forall k^l$$

$$\vdots$$

$$z^{k,L}(x) = g^L(z^{1,L-1}, \ldots, z^{K^{L-1},L-1}), \quad l = L, \quad \forall k^L$$

$$\hat{y}^m = f^m(z^{1,L}, \ldots, z^{K^L,L}), \quad \forall m = 1, \ldots, M$$

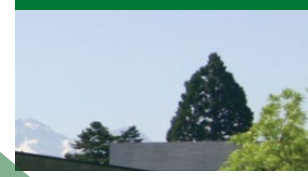


$$z^{k,1}(x) = g^1(\alpha^{k,1} + \sum_{p=1}^{P} x^p \beta^{p,k,1}), \quad l = 1$$

$$z^{k,l}(x) = g^l(\alpha^{n^1} + \sum_{k=1}^{K^{l-1}=1} z^{k,l-1} \beta^{k,l-1}), \quad l > 1$$

$$z^{k,L}(x) = g^L(z^{1,L-1}, ..., z^{K^{L-1},L-1}), \quad l = L$$

$$\hat{y}^m = h^m(z^{1,L}, ..., z^{K^L,L})$$



Background

Neural networks | A2

The major nonlinearity is captured by the layers & neurons

The transformation in the last layer is outcome specific

The transformation into a prediction is outcome- & task- (classification, regression) specific



Background

Neural networks | A3: Coefficients ('weights') & tuning parameters

Huge # of coefficients (called 'weights') to estimate

- # of covariates *x* avg. # of neurons *x* # of layers (minimum)

Many tuning parameters

- # of neurons in specific layers
- # of layers
- Different functions in each layer
- Functions used for generating predictions

Depending on architecture, estimation is computationally very expensive

- May require explicitly designed hardware